TECHNICAL DOCUMENT

EduTech AI Platform

Project Overview & Quick Start

Mono-repo · 7 Services · 75 Tests Passing

PLATFORM	STACK	GENERATED
EduTech v1.0	Java 17 · Spring Boot 3.3.5	March 07, 2026

EduTech AI Platform

Build Passing

Tests 75/75

Java 17

Spring Boot 3.3.5

License MIT

Enterprise-grade, AI-first educational technology platform for coaching centers — delivering adaptive assessments, psychometric profiling, and real-time parent engagement via a React Native APK, backed by a hexagonal microservices architecture.

Table of Contents

1. Project Overview
2. Architecture at a Glance
3. Module Map
4. Tech Stack
5. Quick Start
6. Build Commands
7. Test Summary
8. Architecture Principles

9. Documentation

Project Overview

EduTech AI Platform is an enterprise-grade microservices system designed specifically for coaching centers, parents, and students. Every module is AI-augmented and built to production standards — targeting a 99.99% SLA and sub-100ms API latency on the critical path.

Key capabilities:

- AI-powered adaptive assessment engine using Item Response Theory (IRT) and Computer Adaptive Testing (CAT)
- Psychometric profiling with multi-axis trait mapping (Big Five, RIASEC, learning style fusion)
- Real-time parent transparency dashboard with LLM-generated child performance summaries
- Coaching center operations: scheduling, fee management, attendance, versioned content library
- Secure, token-bound exam mode with AI proctoring (object detection, gaze tracking)
- RAG-powered doubt resolver operating over each coaching center's own content library
- Full mobile delivery via React Native APK with offline-first capability

---

Architecture at a Glance

```
React Native APK
       |  HTTPS
       ▼
┌──────────────────────────────────────────────────────┐
│                   api-gateway :8080                  │
│   Spring Cloud Gateway · JWT RS256 filter            │
│   Redis rate limiter · Resilience4j circuit breaker  │
│   Aggregated OpenAPI / Swagger UI                    │
└──┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──┘
   |      |      |      |      |
   ▼      ▼      ▼      ▼      ▼
auth-svc   center-svc  parent-svc  assess-svc  psych-svc
 :8081       :8083       :8082       :8084       :8085
   |      |      |      |      |
   └──────┴──────┴──────┴──────┴──────┴──────┴──────┴──┘
                              |  Kafka (KRaft)
                              ▼
                     ai-gateway-svc :8086
                     (LLM router / RAG)
                    /          |          \
               Anthropic     OpenAI    psych-ai-svc
               Claude                 (Python FastAPI
                                       sidecar :8095)
```

```
Shared infrastructure (per docker-compose.yml):
  PostgreSQL 16 + pgvector + TimescaleDB
  Redis 7 (session store, rate limiting, token blacklist)
  Kafka 3.6 KRaft — no ZooKeeper
```

All client traffic enters through `api-gateway`. Downstream services are never directly reachable from external clients. Inter-service events flow over Kafka; synchronous service-to-service calls go through `ai-gateway-svc` for LLM operations.

Module Map

📦 Shared Libraries

MODULE	DESCRIPTION	STATUS
`libs/common-security`	JWT RS256 utilities, security filter chain, RBAC helpers shared across all services	COMPLETE ✅
`libs/event-contracts`	Avro schemas for all Kafka event types (auth, audit, notification, domain events)	COMPLETE ✅
`libs/test-fixtures`	Shared test builders, WireMock stubs, Testcontainers configurations	COMPLETE ✅

⚙️ Microservices

MODULE	PORT	DESCRIPTION	STATUS
`services/auth-svc`	8081	Identity & Access Management — registration, login, JWT issuance, OTP/TOTP, Argon2id, hCaptcha	COMPLETE ✅
`services/parent-svc`	8082	Parent profiles, child account links, fee payment records, Firebase push notifications	COMPLETE ✅
`services/center-svc`	8083	Coaching center management: batches, teachers, schedules, fee structures, attendance, content library	COMPLETE ✅
`services/assess-svc`	8084	Exam lifecycle — IRT question bank (pgvector), CAT engine, WebSocket exam sessions, auto-grading	COMPLETE ✅
`services/psych-svc`	8085	Psychometric profiling — Big Five + RIASEC assessments, session history, career path via AI sidecar	COMPLETE ✅
`services/ai-gateway-svc`	8086	Unified LLM router — Anthropic/OpenAI/Ollama dispatch, RAG orchestration, embedding generation	COMPLETE ✅

MODULE	PORT	DESCRIPTION	STATUS
`services/api-gateway`	8080	Spring Cloud Gateway edge — JWT filter, Redis rate limiting, routing, aggregated Swagger UI	COMPLETE ✅

Tech Stack

LAYER	TECHNOLOGY	VERSION
Language	Java	17 (production containers: Temurin 21)
Framework	Spring Boot	3.3.5
Cloud bootstrap	Spring Cloud	2023.0.3
Reactive runtime	Spring WebFlux	(via Spring Cloud BOM)
Database	PostgreSQL + pgvector + TimescaleDB HA	16
Cache	Redis	7
Message bus	Apache Kafka (KRaft — no ZooKeeper)	3.6
JWT	JJWT	0.12.6
Password hashing	Argon2id	(Spring Security 6 built-in)
DB migrations	Flyway	10.17.3
Object mapping	MapStruct (compile-time, zero reflection)	1.5.5.Final
API docs	springdoc-openapi	2.6.0
Architecture enforcement	ArchUnit	1.3.0
HTTP stubs (tests)	WireMock	3.9.1
Event serialization	Apache Avro	1.11.3
SBOM generation	CycloneDX Maven Plugin	2.8.1

Quick Start

Prerequisites

- Java 17+ (Temurin recommended — `sdk install java 17.0.x-tem`)

- Maven 3.9+
- Docker Desktop 4.x (Compose V2 plugin required)
- Git

Steps

```
# 1. Clone
git clone https://github.com/your-org/school-APK.git
cd school-APK

# 2. Configure environment
cp .env.example .env
# Edit .env — at minimum set POSTGRES_ROOT_PASSWORD, REDIS_PASSWORD, and JWT key paths.
# See docs/LOCAL_DEV_GUIDE.md for RSA key generation and field-by-field guidance.

# 3. Start infrastructure (PostgreSQL, Redis, Kafka + dev GUI tools)
cd infrastructure/docker
docker compose --env-file ../../.env up -d
cd ../..

# 4. Compile all modules (shared libs are built first per pom.xml module order)
mvn compile --no-transfer-progress
```

Flyway migrations run automatically on each service's first startup — no manual schema setup is required.

For the full step-by-step guide including RSA key generation, per-service startup, and troubleshooting, see docs/LOCAL_DEV_GUIDE.md.

Build Commands

```
# Compile all modules
mvn compile --no-transfer-progress

# Run all 75 tests across the entire project
mvn test --no-transfer-progress

# Test a single service (--also-make builds its lib dependencies first)
mvn test -pl services/auth-svc --also-make --no-transfer-progress
mvn test -pl services/parent-svc --also-make --no-transfer-progress
mvn test -pl services/center-svc --also-make --no-transfer-progress
mvn test -pl services/assess-svc --also-make --no-transfer-progress
mvn test -pl services/psych-svc --also-make --no-transfer-progress
mvn test -pl services/ai-gateway-svc --also-make --no-transfer-progress
mvn test -pl services/api-gateway --also-make --no-transfer-progress
```

```
# Package a runnable JAR for a single service
mvn package -pl services/auth-svc --also-make -DskipTests --no-transfer-progress

# Override project version (used in CI pipelines)
mvn compile -Drevision=1.2.0
```

Test Summary

SERVICE	TESTS	COVERAGE SCOPE
`auth-svc`	10	Unit (domain + application) + ArchUnit layer boundaries
`center-svc`	11	Unit (domain + application) + ArchUnit layer boundaries
`parent-svc`	11	Unit (domain + application) + ArchUnit layer boundaries
`assess-svc`	11	Unit (domain + application) + ArchUnit layer boundaries
`psych-svc`	12	Unit (domain + application) + ArchUnit layer boundaries
`ai-gateway-svc`	10	Unit (domain + application) + ArchUnit layer boundaries
`api-gateway`	10	Unit (routing + filter) + ArchUnit layer boundaries
Total	**75**	

All tests are deterministic and infrastructure-free. WireMock and in-process stubs replace real HTTP calls. ArchUnit tests enforce hexagonal layer boundaries at build time — domain classes may not import framework packages.

Architecture Principles

PRINCIPLE	IMPLEMENTATION
Hexagonal Architecture	Each service has `domain`, `application`, and `infrastructure` layers. Domain core has zero Spring/JPA/Kafka imports. Ports (interfaces) decouple domain from adapters. ArchUnit tests enforce boundaries at every `mvn test`.
No Lombok	All boilerplate is explicit. Java Records are used for immutable DTOs, command objects, and value types. The codebase stays grep-friendly and IDE-agnostic.
No hardcoded values	Every configurable value lives in `application.yml` backed by an environment variable. No literals appear in business logic or configuration classes.
Soft delete	Entities use a `deleted_at` timestamp. No `DELETE` statement touches business data. Records are recoverable and preserve audit history.

PRINCIPLE	IMPLEMENTATION
Optimistic locking	All aggregate roots carry a JPA `@Version` column. Concurrent-write conflicts surface as `409 Conflict` — no pessimistic locking needed.
RFC 7807 Problem Details	All error responses use `Content-Type: application/problem+json` with `type`, `title`, `status`, `detail`, and `instance` fields (Spring Boot 3 `ProblemDetail`).
JWT RS256	Access tokens are signed with an RSA-2048 private key (held only by `auth-svc`). All other services validate tokens using the public key only — zero shared secrets.
Argon2id	Passwords are hashed with OWASP 2024 recommended parameters: 64 MB memory, 3 iterations, parallelism 2, 16-byte salt, 32-byte hash. All parameters are externalized.
Immutable audit log	Every permission change and LLM interaction is published to a dedicated append-only Kafka topic. Consumers never write to this topic.
All versions in root POM	No version number appears in any child POM `<dependency>` block. All versions are managed in the root `<dependencyManagement>` section — one file, one change.

Documentation

```
docs/
├── adr/                    Architecture Decision Records
├── masterprompt.md         Architecture vision and service specifications
└── LOCAL_DEV_GUIDE.md      Step-by-step local development setup

infrastructure/
└── docker/
    ├── docker-compose.yml  Local infrastructure stack definition
    └── init-db/            PostgreSQL database initialisation scripts

services/{svc}/
└── src/main/resources/
    ├── application.yml     Service configuration (all values from env vars)
    └── db/migration/       Flyway SQL migration scripts
```

- **Local Development Guide**: docs/LOCAL_DEV_GUIDE.md
- **Architecture Decision Records**: docs/adr/
- **Environment Variable Reference**: .env.example